CLASSIFICATION OF HAND-WRITTEN DIGITS

ÁLVARO GALISTEO BERMÚDEZ, DANIEL RATH

Contents

1. Logistic regression for multi-class classification 1
1.1. Our problem 1
1.2. Loss function 2
1.3. Solvability 2
1.4. Regularization 3
2. Stochastic Gradient Descent 5
2.1. Gradient-based methods: cost–accuracy trade-offs 5
3. Results and Discussion 7
3.1. Hyperparameter tuning 7
3.2. Final models and discussion 7
References 9

Date: February 18, 2026.

1. Logistic regression for multi-class classification

Logistic regression is a commonly used model for classification tasks. In our case we want to use logistic regression for multi-class classification, which is a generalization of the binary logistic regression.

In our case, we want to predict the class of a given 8×8 grayscale image of a hand written digit. One would expect that a given input can not be classified as more than one digit at the same time. Therefore, we want to compute the probability of a given input image to belong to every of the 10 classes (digits from 0 to 9) and assign the class with the highest probability to the input image.

1.1. **Our problem.**

An input image is represented as a vector $a_j \in \mathbb{R}^{64}$. Each entry of the vector is an integer between 0 and 16 and corresponds to the intensity of a pixel in the image with 0 being the lowest intensity and 16 being the highest intensity. Each image a_j in the dataset is associated with a label $y_j \in \{0, \dots, 9\}$, which is the digit that is represented in the image. In order to use the labels in our model, we encode them into one-hot vectors $p_j \in \{0,1\}^{10}$, where the i-th entry is 1 if the label is i and 0 otherwise.[Dev]

We want to train a function $\varphi : \mathbb{R}^{64} \times (\mathbb{R}^{64} \times \mathbb{R})^{10} \longrightarrow \mathbb{R}^{10}$ that takes an input image a_j and a parameter $x = (w_i, b_i)_{i=0}^{9}$ and outputs a vector of scores for each class. The score for class i is computed as $\varphi(a_j; x)_i = a_j^T w_i + b_i$. The $w_i \in \mathbb{R}^{64}$ is a weight vector and $b_i \in \mathbb{R}$ is a bias term for class i.

The training of the model consists of the following optimization problem

$$\underset{x \in (\mathbb{R}^{64} \times \mathbb{R})^{10}}{\text{minimize}} \frac{1}{N} \sum_{j=1}^{N} L(p_j, \varphi(a_j; x)), \tag{1.1}$$

with $N \in \mathbb{N}$ being the number of training samples and L the cross entropy loss defined as

$$L(p, s) = \log\Big(\sum_{l=0}^{9} \exp\,(s_l)\Big) - \sum_{i=0}^{9} p_i\, s_i. \tag{1.2}$$

After the model has been trained, we can use it to classify new input images a by computing $\varphi(a; x)$ and assigning the class with the highest score to the input image, i.e.

$$\tilde{y} = \underset{i \in \{0,\dots,9\}}{\arg\max}\ \varphi(a; x)_i.$$

For logistic regression the scores in $\varphi(a_j; x)_i$ are interpreted as probabilities by applying the softmax function, i.e.

$$\sigma(s)_i = \frac{\exp(s_i)}{\sum_{l=0}^{9} \exp(s_l)}. \tag{1.3}$$

Note that in order to classify an input image, we do not need to compute the probabilities $\sigma(\varphi(a; x))_i$ explicitly. This is because the softmax function is monotonically increasing in each of its entries. Therefore, the class with the highest score is also the class with the highest probability. However, we will see in the next section that the softmax function is important for the choice of our loss function.

1.2. **Loss function.**

We want to briefly discuss the choice of the loss function (1.2). For this sections we used [GBC16, Section 6.2.2.3] as a reference. In general, for a given input image a_j we want to predict the probabilities $\sigma(\varphi(a_j;x))_i$ to belong to each of the classes $0 \le i \le 9$.

For this, we want our model to maximize the likelihood to observe the training data. If we assume each training image to be independent, the likelihood that our model predicts the observed training data correctly is given as

$$\mathcal{L}(x) := \prod_{j=1}^{N} \sigma(\varphi(a_j;x))_{y_j}.$$

In order to deal with sums instead of products, we can equivalently minimize

$$-\log \mathcal{L}(x) = -\sum_{j=1}^{N} \log \sigma(\varphi(a_j;x))_{y_j}. \tag{1.4}$$

If we now use the definition of the softmax function (1.3), we can write

$$\begin{aligned} -\log(\sigma(\varphi(a_j;x))_{y_j}) &= -\log\left(\frac{\exp(\varphi(a_j;x)_{y_j})}{\sum_{l=0}^{9}\exp(\varphi(a_j;x)_l)}\right) \\ &= \log\left(\sum_{l=0}^{9}\exp(\varphi(a_j;x)_l)\right) - \varphi(a_j;x)_{y_j}. \end{aligned}$$

The last expression is identical to the loss function in (1.2), when substituting $\varphi(a_j;x)_i$ with s_i. Therefore, minimizing (1.4) yields the same result as solving our optimization problem in (1.1). Also one can see that the choice of the softmax function to get the probabilities from the scores is crucial for the choice of our loss function.

1.3. **Solvability.**

The given optimization problem is a convex problem. This is because the log-sum-exp function is convex as we have already proved in the lecture. For reference one can also check [Boy09, pp. 72, 74]. Also the second term of our loss function (1.2) is linear and therefore convex. Also, φ is an affine mapping in x. And at the same time a convex function over an affine function is also convex (cf. [Boy09, p. 67]).

In general however, the optimization problem is not necessarily solvable. This is because the loss function (1.2) is not strictly convex in its scores s and therefore not strictly convex in the parameter x.

This, is because the loss function (1.2) is invariant under the addition of the same constant to every score s_i. More precisely, for a constant $c \in \mathbb{R}$ we have

$$\log\left(\sum_{l=0}^{9}\exp\left(s_l + c\right)\right) = \log\left(\exp(c)\sum_{l=0}^{9}\exp\left(s_l\right)\right) = \log\left(\sum_{l=0}^{9}\exp\left(s_l\right)\right) + c$$

and also

$$\sum_{i=0}^{9} p_i(s_i + c) = \sum_{i=0}^{9} p_i s_i + c.$$

Therefore, we have

$$L(p, s + c\mathbb{1}) = \log\left(\sum_{l=0}^{9}\exp\left(s_l + c\right)\right) - \sum_{i=0}^{9} p_i(s_i + c) = L(p,s) + c - c = L(p,s).$$

This implies that for a solution x of our optimization problem, we get arbitrary additional solutions, e.g. $x' = (w_i + c\mathbb{1}, b_i)_{i=0}^9$ or $x' = (w_i, b_i + c)_{i=0}^9$ for any constant $c \in \mathbb{R}$. Therefore, the optimization problem is not uniquely solvable.

Even if we ignore non-uniqueness, the minimum in general may fail to be attained at all. A reason is linear separability of the training data. For a parameter x and

$$s_{j,i} := \varphi(a_j; x)_i = a_j^\top w_i + b_i,$$

and $p_{j,y_j} = 1$, we can rewrite the loss as

$$L(p_j, s_j) = \log\left(\sum_{l=0}^{9} e^{s_{j,l}}\right) - s_{j,y_j} = \log\left(1 + \sum_{i \neq y_j} e^{-(s_{j,y_j} - s_{j,i})}\right) \geq 0. \tag{1.5}$$

Hence $L(p_j, s_j)$ is gets small when all the $s_{j,y_j} - s_{j,i}$ are large, e.g. when the correct class score s_{j,y_j} dominates all other scores $s_{j,i}$ by a large margin $\gamma > 0$. If there exists a parameter $\bar{x} = (\bar{w}_i, \bar{b}_i)_{i=0}^9$ such that this is the case, we call the dataset *separable.*

We are then able to scale the parameter $\bar{x}$ with a factor $t > 0$

$$\bar{x}^{(t)} := (t\bar{w}_i, t\bar{b}_i)_{i=0}^9.$$

and get $s_{j,i}^{(t)} = \varphi(a_j; x^{(t)})_i = t\, s_{j,i}$. Thus, for the differences we then get

$$s_{j,y_j}^{(t)} - s_{j,i}^{(t)} = t(s_{j,y_j} - s_{j,i}) \geq t\gamma.$$

If we use this, the above expression (1.5) yields

$$0 \leq L\big(p_j, \varphi(a_j; \bar{x}^{(t)})\big) = \log\left(1 + \sum_{i \neq y_j} e^{-t(s_{j,y_j} - s_{j,i})}\right) \leq \log\left(1 + 9e^{-t\gamma}\right) \xrightarrow[t \to \infty]{} 0.$$

Therefore, if there exists such a separating parameter $\bar{x}$, the optimization term $\sum_{j=1}^N L(p_j, \varphi(a_j; \bar{x}^{(t)}))$ is going to be arbitrarily small for large t, i.e. for large weights and biases $t\bar{w}_i$ and $t\bar{b}_i$. This implies that the minimum of our optimization problem is not attained at a finite parameter x and therefore the optimization problem is not solvable.

Remark 1.3.1. The idea, that linear separability can be a problem for logistic regression, was inspired by [Ada18, Linear Separability and Regularization].

1.4. **Regularization.**

In order to deal with the problem of linear seperability, we can add a regularization term to our optimization problem. For example, we can add an L^2 regularization term $\frac{\lambda}{2}\sum_{i=0}^9 \|w_i\|^2$ with a regularization parameter $\lambda > 0$. This term penalizes large weights w_i and therefore prevents the optimization problem from being unbounded, e.g. in the case of linear separability.

The optimization problem then becomes

$$\underset{x \in (\mathbb{R}^{64} \times \mathbb{R})^{10}}{\text{minimize}} \frac{1}{N} \sum_{j=1}^{N} L(p_j, \varphi(a_j; x)) + \frac{\lambda}{2} \sum_{i=0}^{9} \|w_i\|^2.$$

This optimization problem is now both coercive and strictly convex for fixed biases b_i and therefore uniquely solvable. But it is not strictly convex in the biases b_i and therefore not uniquely solvable in general.

One could now try to add an additional regularization term for the biases b_i to get a strictly convex optimization problem and therefore a uniquely solvable optimization problem for fixed biases b_i. Also a different idea could be to add a constraint to the optimization problem, e.g. $\sum_{i=0}^{9} b_i = 0$, to get rid of the invariance under addition of a constant to all scores s_i as described above. However, we will not go into details here and just stick to the L^2 regularization of the weights w_i.

2. Stochastic Gradient Descent

In the previous section we formulated the training of the multi-class logistic regression model as the finite-sum optimization problem

$$\underset{x\in(\mathbb{R}^{64}\times\mathbb{R})^{10}}{\text{minimize}}\, f(x) := \frac{1}{N}\sum_{j=1}^{N} f_j(x), \qquad f_j(x) := L(p_j, \varphi(a_j; x)). \tag{2.1}$$

Such objectives are standard in statistical learning, since each term corresponds to the loss contributed by one training example. We solve (2.1) using gradient-based methods and focus on stochastic gradient descent (SGD), which leverages the finite-sum structure by replacing the full gradient with inexpensive stochastic estimates.[Bot12]

2.1. Gradient-based methods: cost–accuracy trade-offs.

The gradient of (2.1) decomposes as

$$\nabla f(x) = \frac{1}{N}\sum_{j=1}^{N} \nabla f_j(x). \tag{2.2}$$

This admits several gradient-based strategies whose performance is governed by the trade-off between *per-iteration cost* and the *quality* of the search direction.

Deterministic first-order method: full-batch gradient descent. Full-batch gradient descent uses the exact gradient (2.2):

$$x^{k+1} = x^k - \eta_k \nabla f(x^k). \tag{2.3}$$

Its main advantage is that the update direction is deterministic and consistent across iterations: for sufficiently small step sizes (e.g. $\eta_k \leq 1/L$ when f is L-smooth), one obtains a guaranteed decrease of the objective and stable convergence behavior.[Boy09; Bub15] In convex settings, full-batch gradient descent achieves the standard $O(1/k)$ convergence rate in function value for smooth objectives.[Bub15]

The drawback is computational. Each iteration requires one full pass over all N samples to evaluate $\nabla f(x^k)$, so the per-iteration cost scales linearly with N and can be dominated by memory access and data movement.[Boy09] As a consequence, while the method is sample-efficient (each update uses all available information), it can be time-inefficient for large datasets: progress per iteration can be good, yet progress per unit wall-clock time may be poor when N is large.

A second practical issue is that the method can be sensitive to ill-conditioning: when the problem is poorly conditioned, the step size required for stability can be small, leading to slow progress unless one uses preconditioning or curvature information (e.g. quasi-Newton methods).

Stochastic first-order methods (SGD and mini-batches). SGD replaces $\nabla f(x^k)$ by a stochastic estimator computed from a mini-batch $\mathcal{B}_k$ of size B:

$$g_k = \frac{1}{B}\sum_{j\in\mathcal{B}_k} \nabla f_j(x^k), \qquad x^{k+1} = x^k - \eta_k g_k. \tag{2.4}$$

Under uniform sampling, g_k is an unbiased estimator of the full gradient, i.e. $\mathbb{E}[g_k \mid x^k] = \nabla f(x^k)$, and its variance typically decreases as B increases.[Bot12; Bub15] The key trade-off is that SGD substantially reduces the computational cost per update (from N gradient contributions to B), but introduces gradient noise, which affects stability and asymptotic accuracy.

From an optimization viewpoint, the presence of noise changes the convergence picture. With diminishing step sizes (e.g. $\eta_k \propto 1/\sqrt{k}$ under bounded variance), SGD converges in expectation with a sublinear rate, and with constant step sizes it typically converges only to a neighborhood whose radius depends on the step size and the gradient variance.[Bub15; Bot12] In practice this motivates using step-size schedules (constant during an initial phase and then decreasing) and monitoring performance as a function of *epochs* (passes over the data) rather than iterations alone.[Bot12]

Mini-batches provide an additional lever. Small B yields very cheap, frequent updates but high variance and potentially oscillatory behavior; larger B reduces variance and makes the method behave closer to full-batch gradient descent, but increases the cost per update.[Bot12] Importantly, increasing B does not always translate into proportional speedups in wall-clock time: beyond a certain regime, one may observe diminishing returns because the noise is already sufficiently reduced and additional samples mainly add redundant computation.[SLASFD19] Thus, B should be viewed as a computational hyperparameter that trades off hardware efficiency (vectorization/throughput) against statistical efficiency (noise reduction).[Bot12; SLASFD19]

Overall, full-batch GD tends to be preferable when N is moderate and high-accuracy solutions are required, whereas SGD is often preferable for large-scale training where the objective should decrease quickly in wall-clock time and moderate noise can be tolerated.[Bot12; Bub15]

3. Results and Discussion

In this section we present some of the results of our experiments and discuss their implications. We focus on the comparison between full-batch gradient descent (FGD) and stochastic gradient descent (SGD) in terms of convergence behavior and computational efficiency. Also we analyze the effects of different hyperparameters, such as the learning rate, the mini-batch size and the L^2 regularization parameter on the performance of SGD.

In order to measure our performance we split the data into three different randomly chosen subsets: a training set, a validation set and a test set.Add percentages of these different sets The training set is used to compute the gradients and update the parameters of our model during the training phase, the validation set is used to tune the hyperparameters and to monitor the performance during training, and the test set is used to evaluate the final performance of our model after training.

3.1. Hyperparameter tuning.

In order to find good hyperparameters for our models, we followed the approach to tune them seperately on the validation set. We are aware that this approach is not optimal, since the hyperparameters are not independent of each other and therefore it would be better to tune them jointly, e.g. by doing a grid search over the hyperparameter space. Is this true?

3.1.1. *L^2 regularization.*

First was L^2 reg but why this order?

3.1.2. *Learning rate.*

Compare different leaning rates for both FGD and SGD:

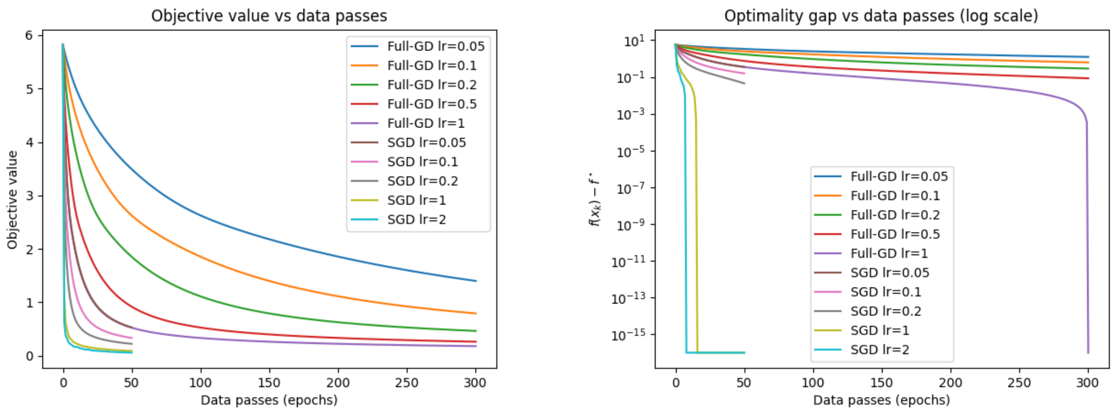

Figure 1. Objective value and optimality gap vs. epoch for different learning rates

3.1.3. *Batch size.*

3.2. Final models and discussion.

Finally, we want to compare both our best final models, i.e. our models that achieved the best accuracy on the validation set. We can see that both our models perform fairly well on the test sets, by looking at the confusion matrices (figure 2).

Although both models perform very well, the SGD model converges much faster than the FGD model (cf. figure 3), which is a common observation in practice. This is because SGD updates more often and can therefore make faster progress in the early stages of training, while FGD requires more time to compute the full gradient and update the parameters.

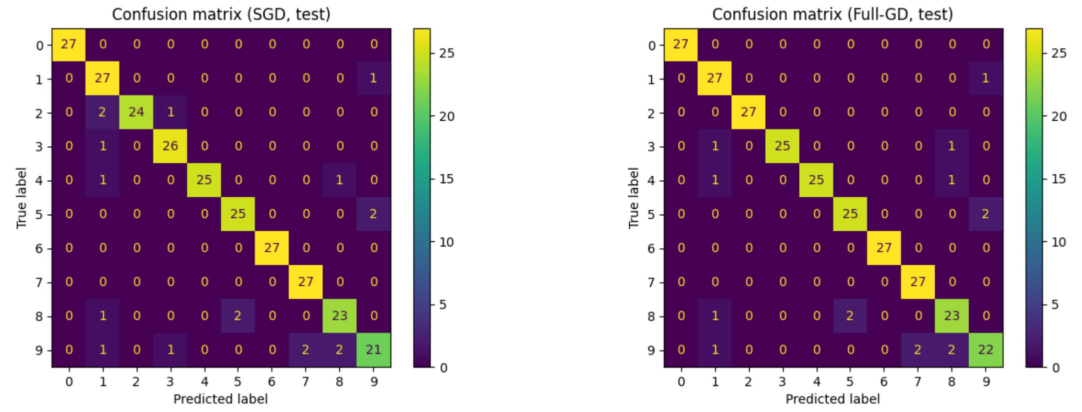

FIGURE 2. Confusion matrices for SGD and FGD on the test set

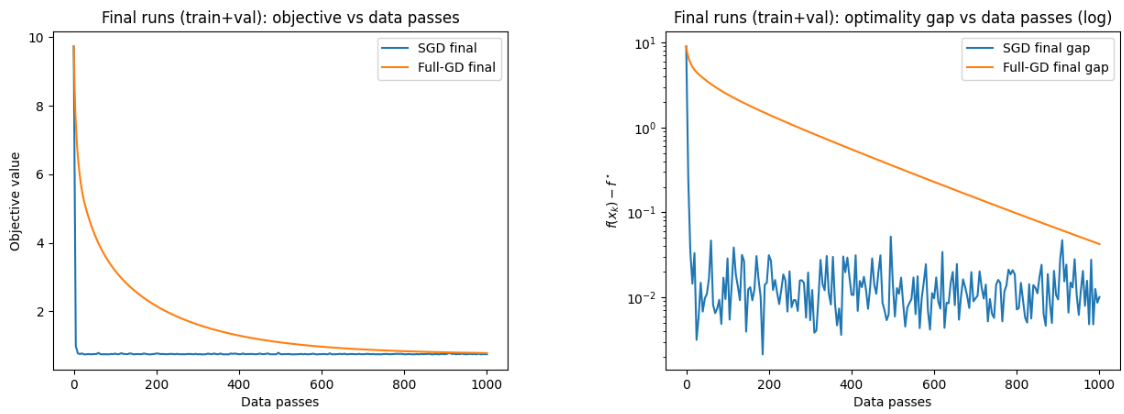

FIGURE 3. Objective value and optimality gap vs. data passes for SGD and FGD on the validation set

However, FGD can achieve a better and more consistent final accuracy than SGD if we allow it to run for enough epochs, since it uses the exact gradient and can therefore make more consistent progress towards the optimal solution. This can be seen very well in the optimality gap plot, since it has a log scale and therefore the oscillations of SGD are visible.

Overall, the choice between FGD and SGD depends on the specific problem and the computational resources available. For large datasets and limited computational resources SGD is often the only choice because FGD would take to much resources.

References

[Ada18] Ryan Adams. *Linear Classification with Logistic Regression. Lecture notes.* COS 324 – Elements of Machine Learning. Princeton University. Oct. 8, 2018. URL: https://www.cs.princeton.edu/courses/archive/spring19/cos324/files/logistic-regression.pdf (visited on 02/10/2026).

[Bot12] Léon Bottou. "Stochastic Gradient Descent Tricks". In: *Neural Networks: Tricks of the Trade.* Ed. by Grégoire Montavon, Geneviève B. Orr, and Klaus-Robert Müller. Vol. 7700. Lecture Notes in Computer Science. Springer, 2012, pp. 421–436.

[Boy09] Stephen P. Boyd. *Convex optimization.* Ed. by Lieven Vandenberghe. 7th ed. first published 2004. Cambridge University Press, 2009. 716 pp. ISBN: 9780521833783.

[Bub15] Sébastien Bubeck. *Convex Optimization: Algorithms and Complexity.* Foundations and Trends® in Machine Learning. FnT in Machine Learning, 8(3-4):231–357. 2015.

[Dev] scikit-learn Developers. *Documentation of scikit-learn Machine Learning in Python.* Version 1.8.0 (stable). URL: https://scikit-learn.org (visited on 02/09/2026).

[GBC16] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning.* http://www.deeplearningbook.org. MIT Press, 2016.

[SLASFD19] Christopher J. Shallue, Jaehoon Lee, Joseph Antognini, Jascha Sohl-Dickstein, Roy Frostig, and George E. Dahl. "Measuring the Effects of Data Parallelism on Neural Network Training". In: *Journal of Machine Learning Research* 20.112 (2019), pp. 1–49.